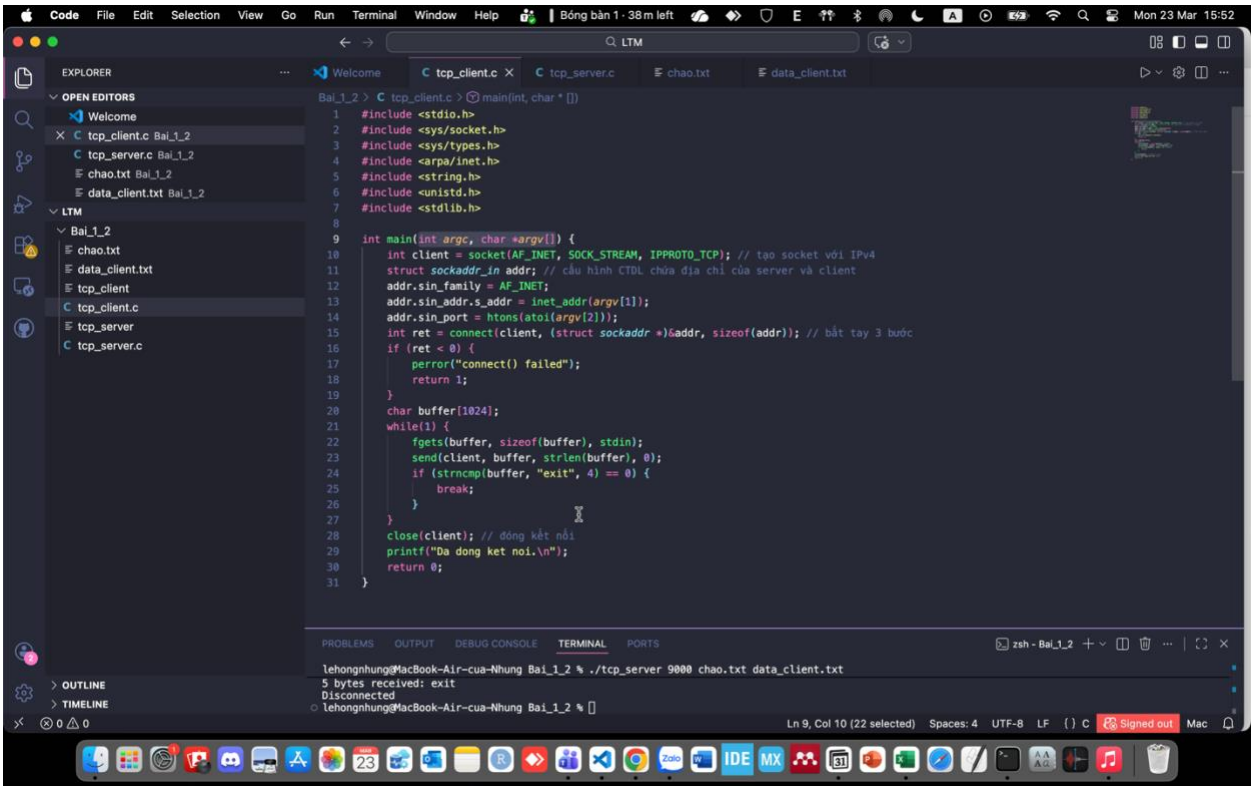
BÁO CÁO BÀI TẬP NGÀY 17/03/2026
HỌC PHẦN: LẬP TRÌNH MẠNG

Sinh viên: Lê Hồng Nhung
MSSV: 20235393

Bài 1: Viết chương trình tcp_client, kết nối đến máy chủ xác định bởi IP và cổng. Nhận dữ liệu từ bàn phím và gửi tới server.



Hình 1: Chương trình tcp_client.

Bài 2: Viết chương trình tcp_server, đợi kết nối ở cổng. Mỗi khi có client kết nối, xâu chào được in ra lưu trong file xác định. Ghi toàn bộ nội dung client gửi đến vào 1 file.

Code File Edit Selection View Go Run Terminal Window Help Bóng bàn 1 - 38m left Mon 23 Mar 15:52

LTM

EXPLORER

OPEN EDITORS
- Welcome
- tcp_client.c Bai_1_2
- tcp_server.c Bai_1_2
- chao.txt Bai_1_2
- data_client.txt Bai_1_2

LTM
- Bai_1_2
  - chao.txt
  - data_client.txt
  - tcp_client
  - tcp_client.c
  - tcp_server
  - tcp_server.c

Welcome | tcp_client.c | tcp_server.c | chao.txt | data_client.txt

Bai_1_2 > tcp_server.c > main(int, char * [])

```c
#include <stdio.h>
#include <stdlib.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <string.h>

int main(int argc, char *argv[]) {
    int listener = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);

    struct sockaddr_in addr;
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
    addr.sin_port = htons(atoi(argv[1]));

    int opt = 1;
    // Thiết lập SO_REUSEADDR để giải phóng cổng ngay khi server tắt
    if (setsockopt(listener, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt)) < 0) {
        perror("setsockopt failed");
        exit(EXIT_FAILURE);
    }

    int ret = bind(listener, (struct sockaddr *)&addr, sizeof(addr));
    if (ret < 0) {
        perror("bind() failed");
        exit(1);
    }

    ret = listen(listener, 5);
    if (ret < 0) {
        perror("listen() failed");
        exit(1);
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
lehongnhung@MacBook-Air-cua-Nhung Bai_1_2 % ./tcp_server 9000 chao.txt data_client.txt
5 bytes received: exit
Disconnected
lehongnhung@MacBook-Air-cua-Nhung Bai_1_2 %
```

OUTLINE
TIMELINE

Ln 74, Col 22 Spaces: 4 UTF-8 LF {} C Signed out Mac

Code File Edit Selection View Go Run Terminal Window Help Bóng bàn 1 - 38m left Mon 23 Mar 15:52

LTM

EXPLORER

OPEN EDITORS
- Welcome
- tcp_client.c Bai_1_2
- tcp_server.c Bai_1_2
- chao.txt Bai_1_2
- data_client.txt Bai_1_2

LTM
- Bai_1_2
  - chao.txt
  - data_client.txt
  - tcp_client
  - tcp_client.c
  - tcp_server
  - tcp_server.c

Welcome | tcp_client.c | tcp_server.c | chao.txt | data_client.txt

Bai_1_2 > tcp_server.c > main(int, char * [])

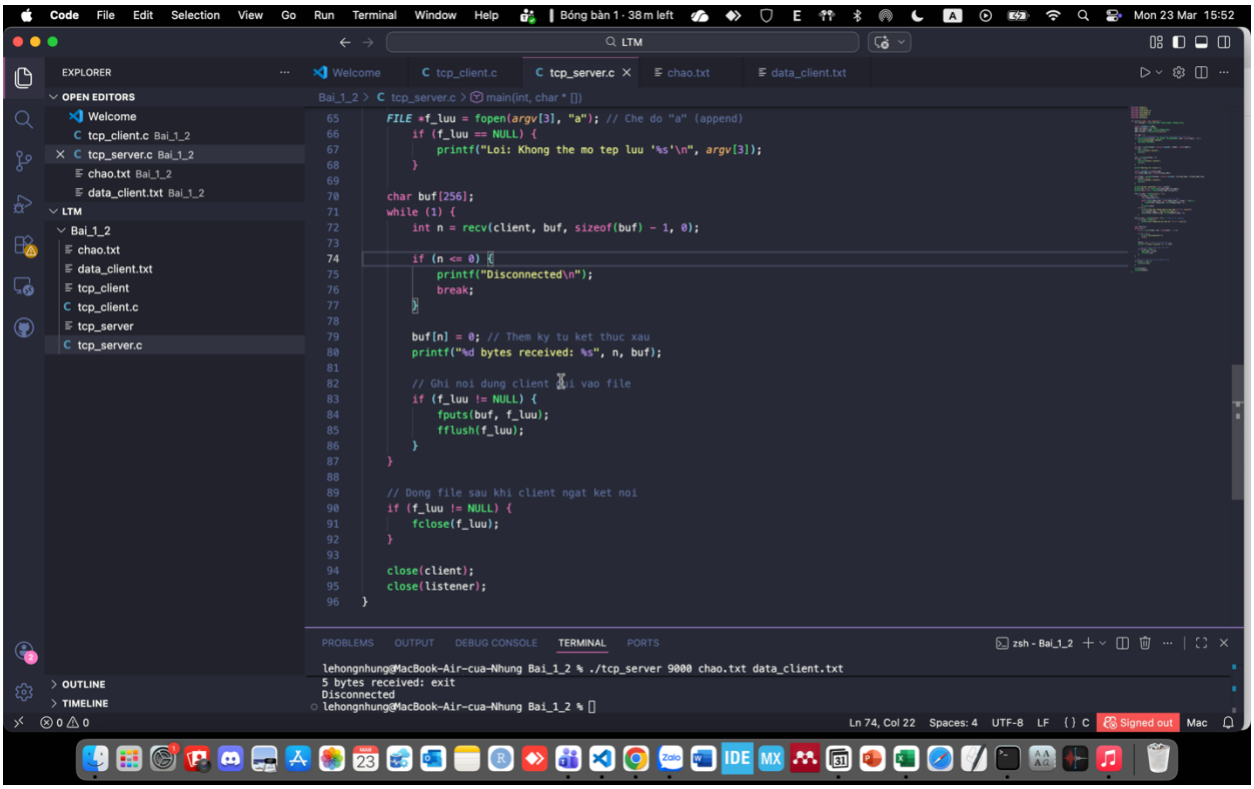
```c
    printf("Waiting for client\n");

    struct sockaddr_in client_addr;
    int client_addr_len = sizeof(client_addr);

    int client = accept(listener, (struct sockaddr *)&client_addr, &client_addr_len);
    if (client < 0) {
        perror("accept() failed");
        exit(1);
    }

    printf("Client connected: %d\n", client);
    printf("IP: %s\n", inet_ntoa(client_addr.sin_addr));
    printf("Port: %d\n", ntohs(client_addr.sin_port));

    FILE *f_chao = fopen(argv[2], "r");
        if (f_chao != NULL) {
            char dong_chao[256];
            // Doc tung dong trong tep cho den het (EOF)
            while (fgets(dong_chao, sizeof(dong_chao), f_chao) != NULL) {
                send(client, dong_chao, strlen(dong_chao), 0);
            }
            fclose(f_chao);
        } else {
            printf("Canh bao: Khong the mo tep chao '%s'\n", argv[2]);
            char default_msg[] = "Server xin chao!\n";
            send(client, default_msg, strlen(default_msg), 0);
        }

    FILE *f_luu = fopen(argv[3], "a"); // Che do "a" (append)
        if (f_luu == NULL) {
            printf("Loi: Khong the mo tep luu '%s'\n", argv[3]);
        }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

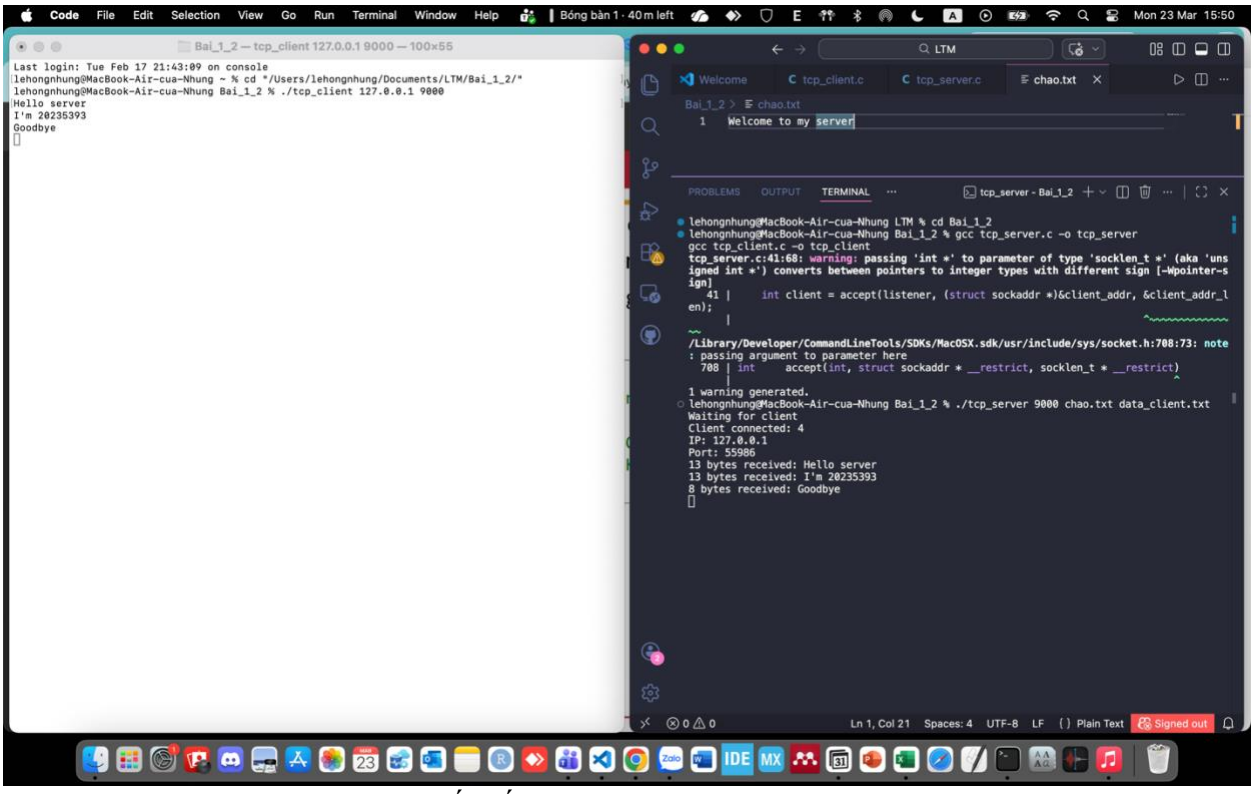
```
lehongnhung@MacBook-Air-cua-Nhung Bai_1_2 % ./tcp_server 9000 chao.txt data_client.txt
5 bytes received: exit
Disconnected
lehongnhung@MacBook-Air-cua-Nhung Bai_1_2 %
```

OUTLINE
TIMELINE

Ln 74, Col 22 Spaces: 4 UTF-8 LF {} C Signed out Mac

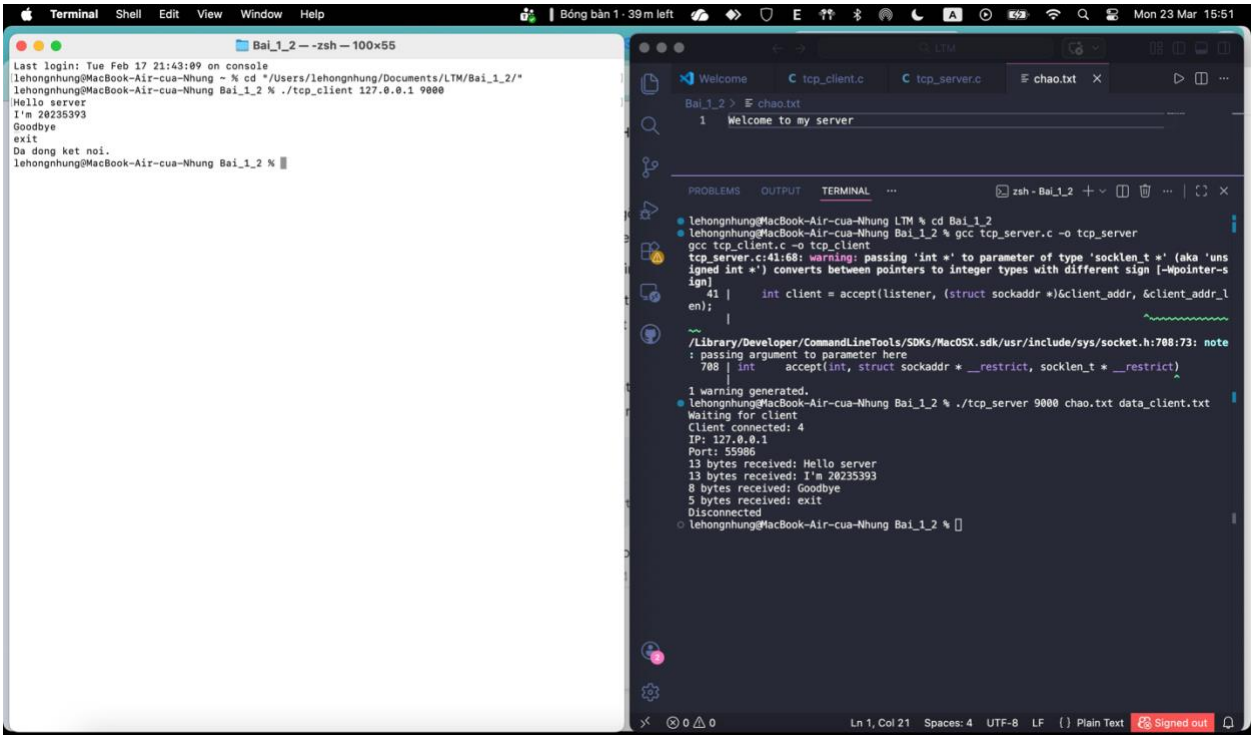


Hình 2: Chương trình tcp_server.



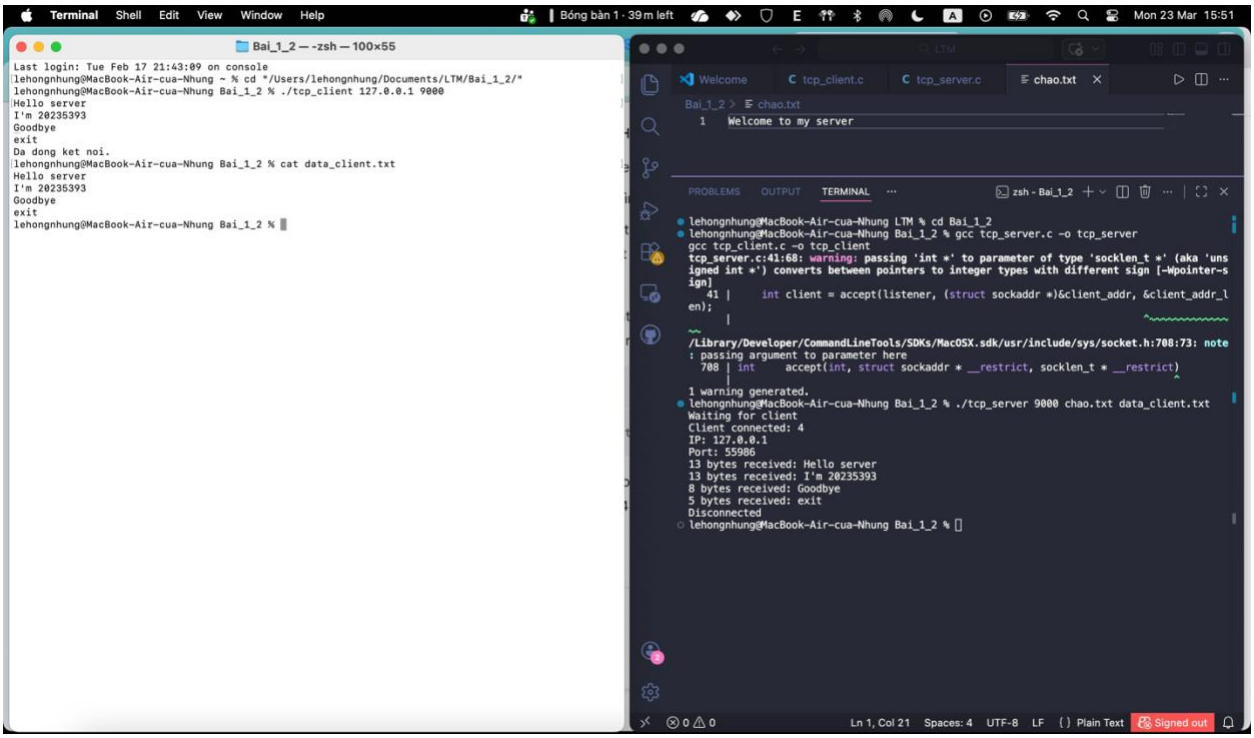
Hình 3. Kết nối client - server và gửi lời chào.

Giải thích: Terminal bên trái đại diện cho client, bên phải đại diện cho server. Ở terminal client, câu lệnh ./tcp_client 127.0.0.1 9000 thực hiện chương trình tcp_client với 2 tham số truyền vào là 127.0.0.1 (localhost) và 9000 (cổng), kết nối tới server. Khi gõ vào terminal client các dòng Hello server, I'm 20235393, Goodbye; tương ứng bên terminal server sẽ hiện ra các dòng đó.



Hình 4: Đóng kết nối.

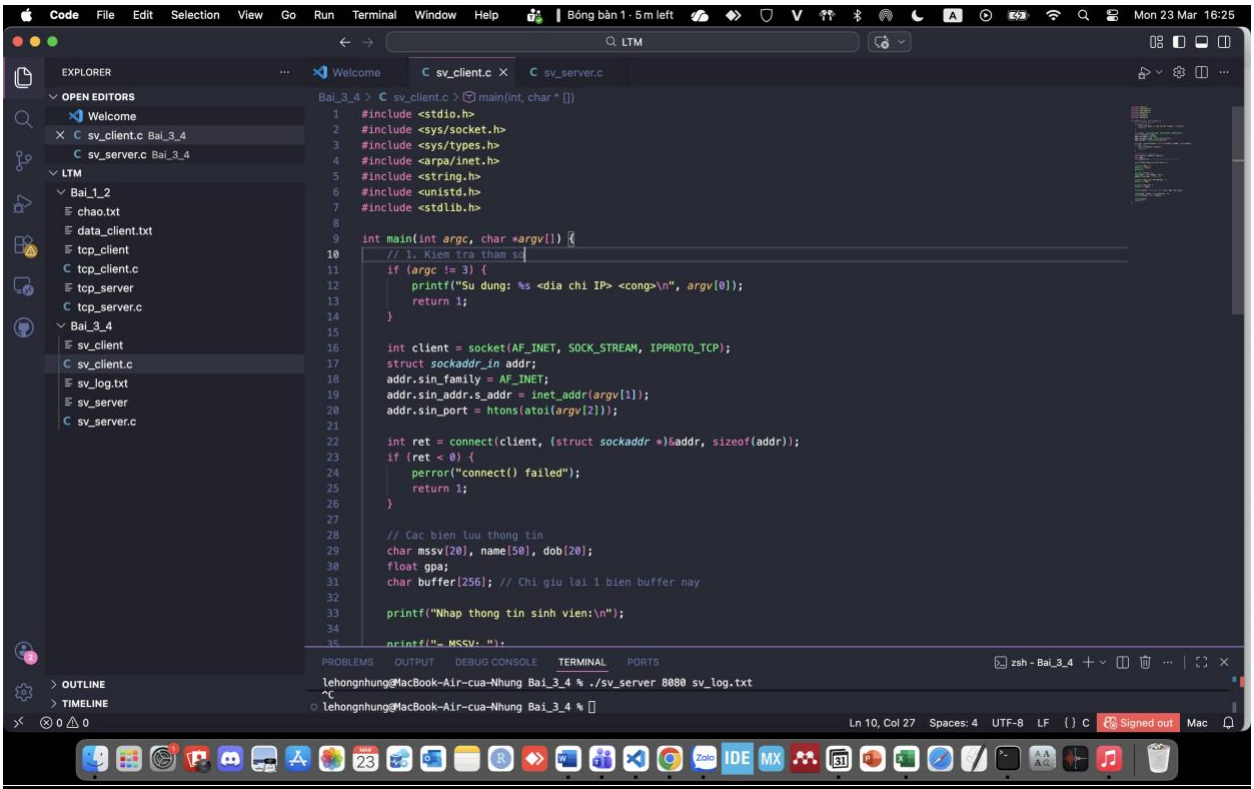
Giải thích: Khi gõ "exit" ở terminal client, đây là tín hiệu đóng kết nối đã được thiết lập trong đoạn code, nên kết nối sẽ đóng lại.

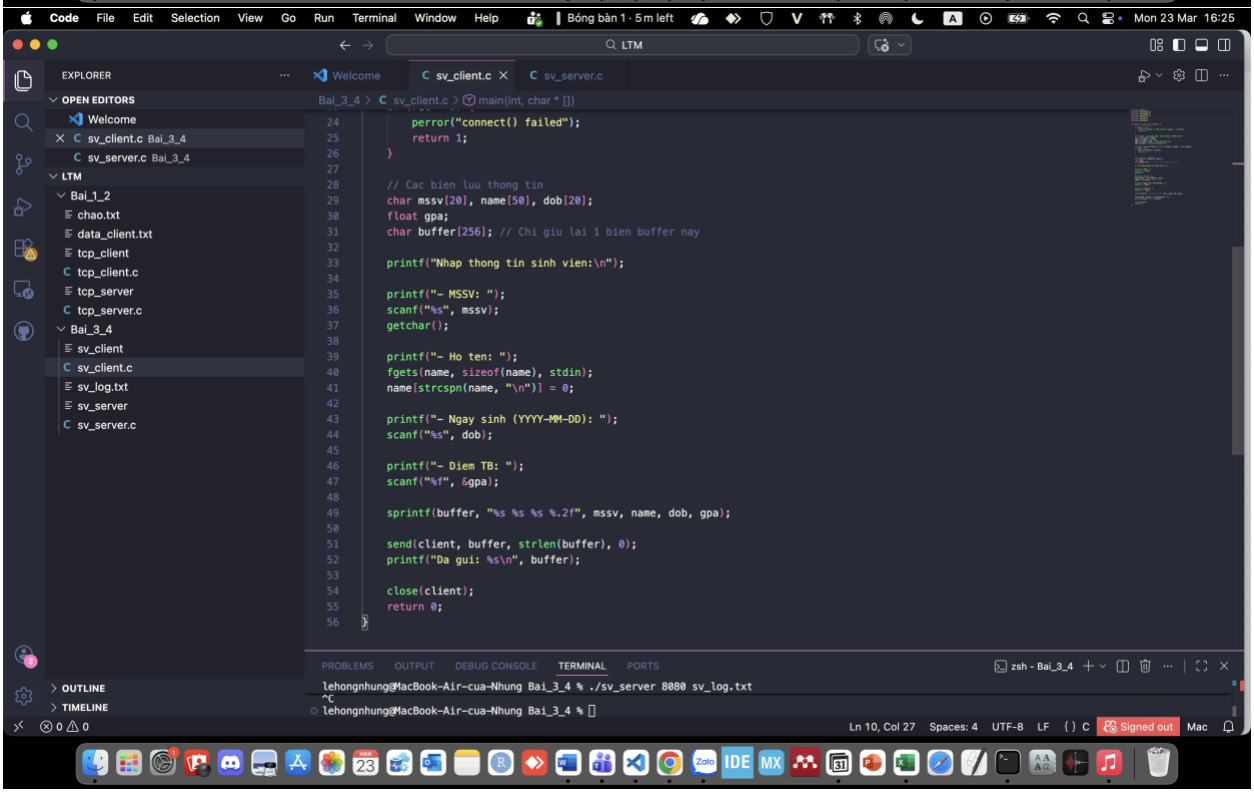


Hình 5: File lưu các xâu chào client đã gửi.

Giải thích: Thực hiện câu lệnh cat data_client.txt để xem nội dung file. Trong file lưu các xâu từng được client gửi qua server.

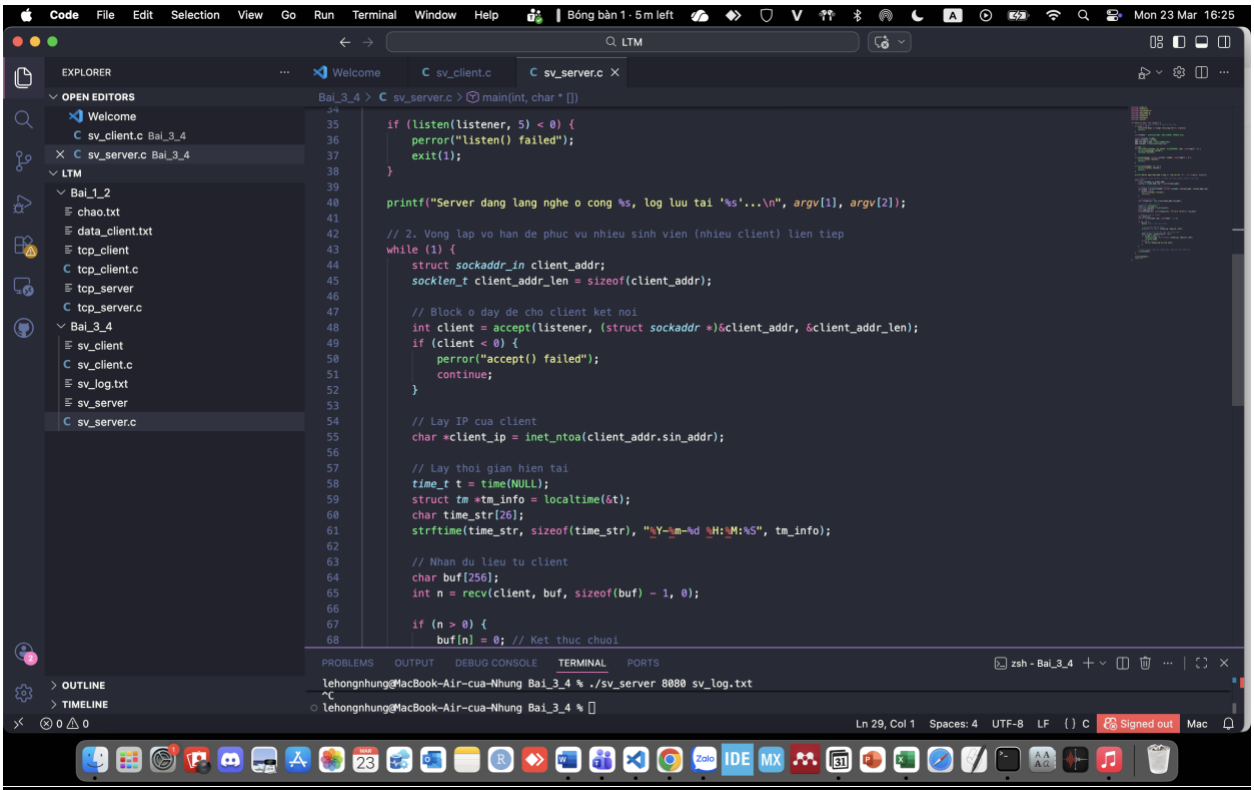
Bài 3: Viết chương trình sv_client, gửi thông tin MSSV, Họ tên, Ngày sinh, Điểm GPA.





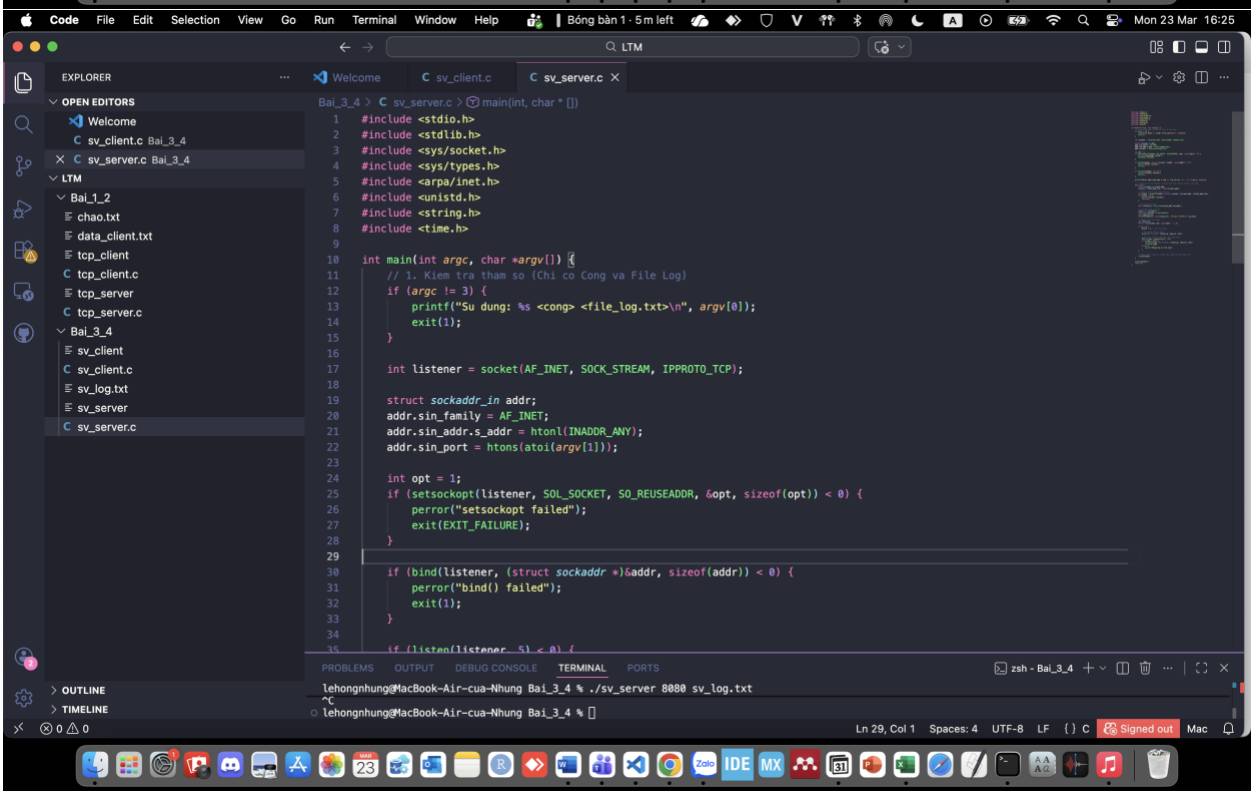
Hình 6: Chương trình sv_client.

Bài 4: Viết chương trình sv_server, nhận thông tin từ client.



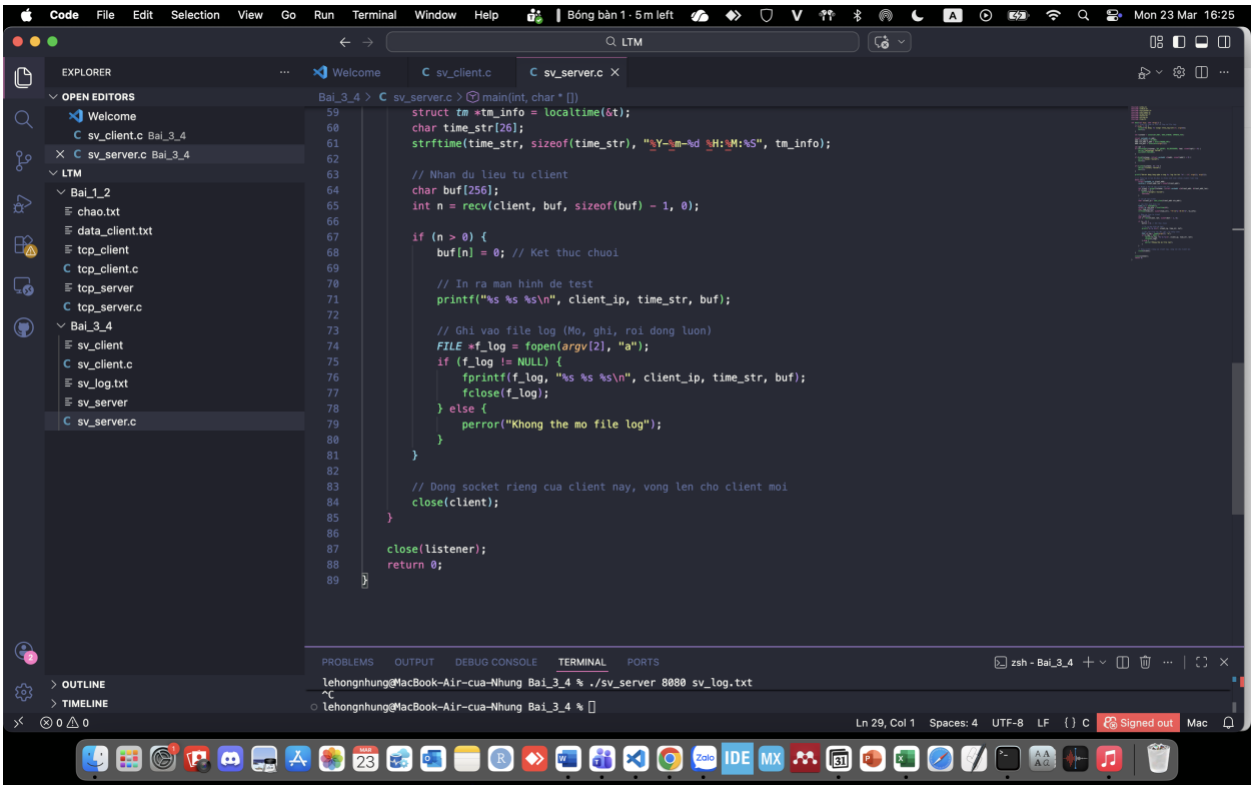
```c
    if (listen(listener, 5) < 0) {
        perror("listen() failed");
        exit(1);
    }

    printf("Server dang lang nghe o cong %s, log luu tai '%s'...\n", argv[1], argv[2]);

    // 2. Vong lap vo han de phuc vu nhieu sinh vien (nhieu client) lien tiep
    while (1) {
        struct sockaddr_in client_addr;
        socklen_t client_addr_len = sizeof(client_addr);

        // Block o day de cho client ket noi
        int client = accept(listener, (struct sockaddr *)&client_addr, &client_addr_len);
        if (client < 0) {
            perror("accept() failed");
            continue;
        }

        // Lay IP cua client
        char *client_ip = inet_ntoa(client_addr.sin_addr);

        // Lay thoi gian hien tai
        time_t t = time(NULL);
        struct tm *tm_info = localtime(&t);
        char time_str[26];
        strftime(time_str, sizeof(time_str), "%Y-%m-%d %H:%M:%S", tm_info);

        // Nhan du lieu tu client
        char buf[256];
        int n = recv(client, buf, sizeof(buf) - 1, 0);

        if (n > 0) {
            buf[n] = 0; // Ket thuc chuoi
```

```
lehongnhung@MacBook-Air-cua-Nhung Bai_3_4 % ./sv_server 8080 sv_log.txt
^C
lehongnhung@MacBook-Air-cua-Nhung Bai_3_4 %
```

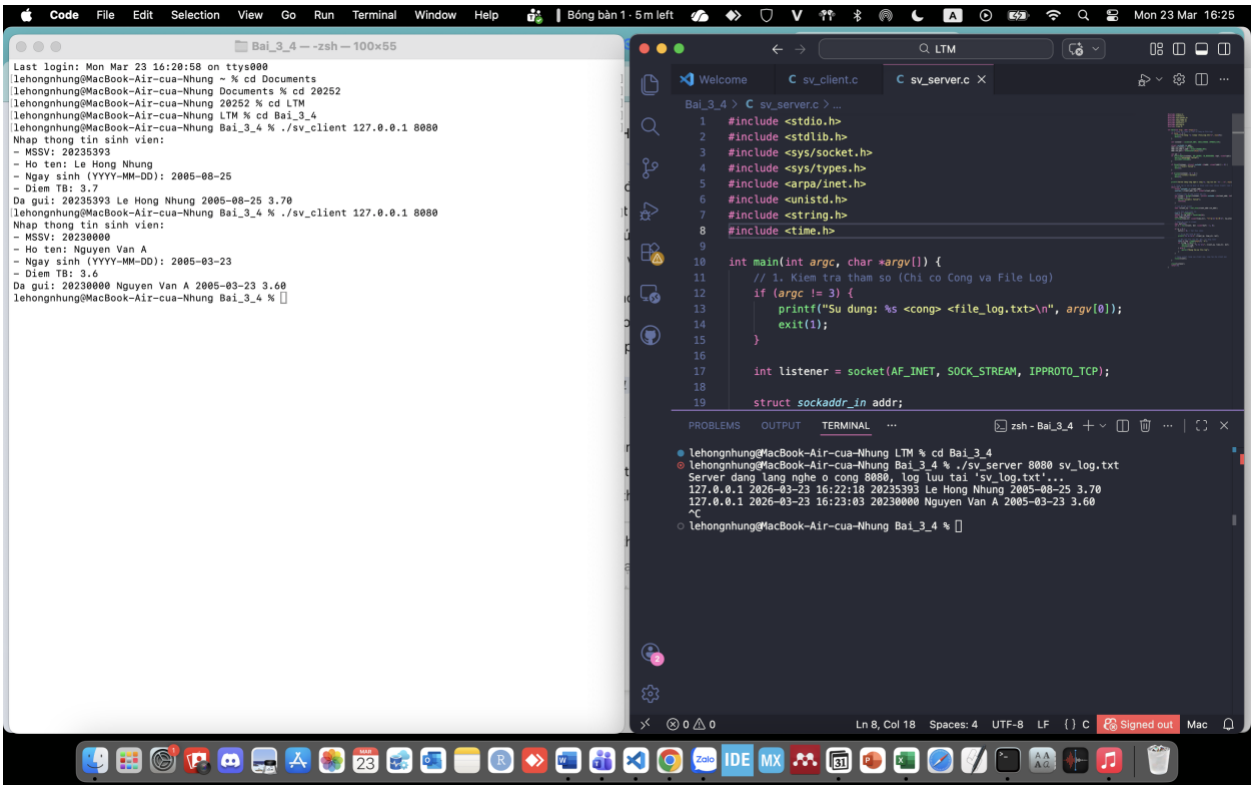


```c
#include <stdio.h>
#include <stdlib.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <string.h>
#include <time.h>

int main(int argc, char *argv[]) {
    // 1. Kiem tra tham so (Chi co Cong va File Log)
    if (argc != 3) {
        printf("Su dung: %s <cong> <file_log.txt>\n", argv[0]);
        exit(1);
    }

    int listener = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);

    struct sockaddr_in addr;
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
    addr.sin_port = htons(atoi(argv[1]));

    int opt = 1;
    if (setsockopt(listener, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt)) < 0) {
        perror("setsockopt failed");
        exit(EXIT_FAILURE);
    }

    if (bind(listener, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
        perror("bind() failed");
        exit(1);
    }

    if (listen(listener, 5) < 0) {
```

```
lehongnhung@MacBook-Air-cua-Nhung Bai_3_4 % ./sv_server 8080 sv_log.txt
^C
lehongnhung@MacBook-Air-cua-Nhung Bai_3_4 %
```



Hình 7: Chương trình sv_server.



Hình 8: Gửi thông tin sinh viên từ terminal client (bên trái) tới server (bên phải).

Giải thích: Sau khi gửi thông tin, client tự động đóng kết nối, nên nếu muốn gửi thêm thông tin thì chạy thêm một lần câu lệnh ./sv_client 127.0.0.1 8080. Khi thông tin đầy đủ được gửi đi, bên terminal server sẽ tự động hiện thông tin đó.